Test Integral Temas

Tema: Crema & Naranja

Materia: Pruebas de Conversión

Integrantes: Conversor de Temas

> *Archivo de prueba extrema para validar el conversor **md2pdf.py** Cubre: títulos, formato, listas, imágenes, tablas, citas, código y LaTeX.*

---

1. Títulos (H1 a H6)

H1 — Título principal

H2 — Sección

H3 — Subsección

H4 — Detalle

H5 — Sub-detalle

H6 — Nota al pie

---

2. Formato de texto

Párrafo normal con **negritas**, *cursivas*, ***ambas***, ~~tachado~~ y `código inline`.

Enlace a OpenAI

También enlaces sin protocolo: admin@example.com

Línea con sub$_{\text{índice}}$ y super^{índice} (según soporte).

---

3. Listas

3.1 Desordenada

- Item A
- Item B
  - Subitem B.1
    - Subitem B.1.1
    - Subitem B.1.2
- Item C

3.2 Ordenada

1. Primero
2. Segundo
   1. Sub segundo 1
   2. Sub segundo 2
3. Tercero

3.3 Mixta anidada

- Lenguajes
  1. Python
  2. JavaScript
  3. Rust
- Frameworks
  - Django
  - React
    - Hooks
    - Context
  - Axum

---

4. Imágenes





---

5. Tablas

5.1 Tabla simple

Producto	Precio	Stock
Widget Alpha	$12.50	142
Widget Beta	$24.00	89
Widget Gamma	$8.75	356

5.2 Tabla compleja con alineación

Característica	TypeScript	Rust	Python
Tipado estático	✅	✅	❌
Compilado	❌	✅	❌
Garbage Collector	✅	❌	✅
Pattern Matching	❌	✅	❌
Async nativo	✅	✅	✅

5.3 Tabla con celdas multilínea y formateo interno

Módulo	Versión	Notas
`rich`	13.0+	Panel, Table, Syntax — *menús en terminal*
`pygments`	2.17+	**monokai** style para resaltado de código
`weasyprint`	60+	Fallback PDF si Puppeteer no está disponible

---

6. Blockquotes

> *Esto es una cita normal.*
>
> ***Nota:** Las citas pueden contener formato y* `código`*.*
>
> *— Anónimo*
>
> *Nivel 1*
>
> > *Nivel 2*
> >
> > > *Nivel 3 con **negrita** y* `inline`

---

7. Bloques de código

7.1 Python

```python
#!/usr/bin/env python3
import asyncio
from dataclasses import dataclass
from typing import Optional


@dataclass
class Config:
    host: str = "localhost"
    port: int = 8080
    debug: bool = False
    api_key: Optional[str] = None


async def fetch_data(url: str, timeout: float = 10.0) -> dict:
    """Fetch JSON data from a URL with timeout."""
    import aiohttp

    async with aiohttp.ClientSession() as session:
        async with session.get(url, timeout=aiohttp.ClientTimeout(total=timeout))
as resp:
            resp.raise_for_status()
            data = await resp.json()
            return data


async def main():
    cfg = Config(host="0.0.0.0", port=3000, debug=True)
    print(f"Server starting on {cfg.host}:{cfg.port}")

    try:
        result = await fetch_data("https://api.example.com/data")
        print(f"Received {len(result)} items")
    except asyncio.TimeoutError:
        print("Request timed out")
    except Exception as e:
        print(f"Error: {e}")


if __name__ == "__main__":
    asyncio.run(main())
```

7.2 JavaScript / Node.js

```
import { createServer } from "node:http";
import { readFile } from "node:fs/promises";
import { join, extname } from "node:path";

const MIME_TYPES = {
  ".html": "text/html; charset=utf-8",
  ".css": "text/css",
  ".js": "application/javascript",
  ".json": "application/json",
  ".png": "image/png",
  ".svg": "image/svg+xml",
};

const server = createServer(async (req, res) => {
  try {
    const url = new URL(req.url, `http://${req.headers.host}`);
    let filePath = join(".", url.pathname === "/" ? "index.html" : url.pathname);
    const ext = extname(filePath);
    const mime = MIME_TYPES[ext] || "application/octet-stream";

    const content = await readFile(filePath);
    res.writeHead(200, { "Content-Type": mime });
    res.end(content);
  } catch (err) {
    if (err.code === "ENOENT") {
      res.writeHead(404);
      res.end("Not found");
    } else {
      res.writeHead(500);
      res.end("Internal error");
    }
  }
});

const PORT = process.env.PORT || 3000;
server.listen(PORT, () => {
  console.log(`🚀 Server running on http://localhost:${PORT}`);
});
```

7.3 HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Dashboard</title>
  <link rel="stylesheet" href="styles.css" />
  <style>
    :root {
      --primary: #6366f1;
      --bg: #0f111a;
      --text: #e2e8f0;
    }

    * { box-sizing: border-box; margin: 0; padding: 0; }

    body {
      background: var(--bg);
      color: var(--text);
      font-family: 'Inter', sans-serif;
      display: grid;
      grid-template-columns: 260px 1fr;
      min-height: 100vh;
    }

    aside {
      border-right: 1px solid #1e2030;
      padding: 2rem;
    }

    aside nav a {
      display: block;
      color: var(--text);
      padding: 0.5rem 0;
      text-decoration: none;
      border-radius: 6px;
      transition: background 0.2s;
    }

    aside nav a:hover {
      background: rgba(99, 102, 241, 0.1);
    }

    main {
      padding: 2rem 3rem;
    }

    .card {
      background: #161822;
      border: 1px solid #1e2030;
      border-radius: 12px;
      padding: 1.5rem;
      margin-bottom: 1.5rem;
    }

    button {
      background: var(--primary);
      color: #fff;
      border: none;
      padding: 0.6rem 1.2rem;
```

```
      border-radius: 8px;
      font-weight: 600;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <aside>
    <h2>App</h2>
    <nav>
      <a href="/">Inicio</a>
      <a href="/analytics">Analytics</a>
      <a href="/settings">Settings</a>
    </nav>
  </aside>
  <main>
    <div class="card">
      <h1>Dashboard</h1>
      <p>Welcome back, <strong>Admin</strong>.</p>
      <button onclick="refresh()">Refresh Data</button>
    </div>
  </main>
</body>
</html>
```

7.4 Bash / Shell

```bash
#!/usr/bin/env bash
set -euo pipefail

PROJECT="md2pdf"
VERSION="2.0.0"
BUILD_DIR="./dist"

info()  { printf "[1;34m[INFO][0m %s
" "$*"; }
ok()    { printf "[1;32m[ OK ][0m %s
" "$*"; }
err()   { printf "[1;31m[ERROR][0m %s
" "$*" >&2; exit 1; }

clean() {
  info "Limpiando directorio de build …"
  rm -rf "$BUILD_DIR"
  mkdir -p "$BUILD_DIR"
}

build() {
  info "Compilando ${PROJECT} v${VERSION} …"
  pip install -r requirements.txt --quiet
  npm ci --silent || npm install --silent
  ok "Dependencias listas"
}

test() {
  info "Ejecutando tests …"
  python3 -m pytest tests/ -v --tb=short 2>&1 | tail -20
  ok "Tests pasaron"
}

package() {
  info "Empaquetando …"
  tar -czf "${BUILD_DIR}/${PROJECT}-${VERSION}.tar.gz" \
    --exclude=node_modules \
    --exclude=__pycache__ \
    --exclude=.pytest_cache \
    .
  ok "Package → ${BUILD_DIR}/${PROJECT}-${VERSION}.tar.gz"
}

case "${1:-help}" in
  clean)   clean ;;
  build)   build ;;
  test)    test ;;
  package) package ;;
  all)     clean && build && test && package ;;
  help|*)  echo "Uso: $0 {clean|build|test|package|all}" ;;
esac
```

7.5 Rust

```rust
use std::collections::HashMap;
use std::fs;
use std::io::{self, BufRead};
use std::path::Path;

#[derive(Debug, Clone)]
struct Task {
    id: u64,
    title: String,
    done: bool,
    tags: Vec<String>,
}

impl Task {
    fn new(id: u64, title: &str) -> Self {
        Self {
            id,
            title: title.to_string(),
            done: false,
            tags: Vec::new(),
        }
    }

    fn toggle(&mut self) {
        self.done = !self.done;
    }

    fn add_tag(&mut self, tag: &str) {
        if !self.tags.contains(&tag.to_string()) {
            self.tags.push(tag.to_string());
        }
    }
}

fn load_tasks<P: AsRef<Path>>(path: P) -> io::Result<Vec<Task>> {
    let file = fs::File::open(path)?;
    let reader = io::BufReader::new(file);
    let mut tasks = Vec::new();

    for (i, line) in reader.lines().enumerate() {
        let line = line?;
        let trimmed = line.trim();
        if !trimmed.is_empty() {
            tasks.push(Task::new(i as u64, trimmed));
        }
    }
    Ok(tasks)
}

fn group_by_tag(tasks: &[Task]) -> HashMap<String, Vec<&Task>> {
    let mut map: HashMap<String, Vec<&Task>> = HashMap::new();
    for task in tasks {
        if task.tags.is_empty() {
            map.entry("untagged".into()).or_default().push(task);
        } else {
            for tag in &task.tags {
                map.entry(tag.clone()).or_default().push(task);
            }
        }
    }
```

```
    map
}

fn main() -> io::Result<()> {
    let tasks = load_tasks("tasks.txt")?;
    println!("Loaded {} tasks", tasks.len());

    let grouped = group_by_tag(&tasks);
    for (tag, items) in &grouped {
        println!("  [{tag}] {} items", items.len());
    }
    Ok(())
}
```

7.6 SQL

```
-- Sample: e-commerce schema
CREATE TABLE IF NOT EXISTS users (
    id         SERIAL PRIMARY KEY,
    email      VARCHAR(255) NOT NULL UNIQUE,
    name       VARCHAR(120) NOT NULL,
    created_at TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_users_email ON users (email);

CREATE TABLE IF NOT EXISTS orders (
    id         SERIAL    PRIMARY KEY,
    user_id    INTEGER   NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    total      NUMERIC(10,2) NOT NULL CHECK (total >= 0),
    status     VARCHAR(20) NOT NULL DEFAULT 'pending'
                   CHECK (status IN ('pending','paid','shipped','cancelled')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS order_items (
    id        SERIAL  PRIMARY KEY,
    order_id  INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    sku       VARCHAR(50) NOT NULL,
    qty       INTEGER     NOT NULL CHECK (qty > 0),
    price     NUMERIC(10,2) NOT NULL
);

-- Complex query: top users by revenue
SELECT
    u.id,
    u.name,
    COUNT(o.id)               AS order_count,
    SUM(o.total)::NUMERIC(10,2) AS revenue
FROM users u
JOIN orders o ON o.user_id = u.id
WHERE o.status = 'paid'
  AND o.created_at >= NOW() - INTERVAL '30 days'
GROUP BY u.id, u.name
HAVING SUM(o.total) > 100
ORDER BY revenue DESC
LIMIT 10;
```

8. Líneas horizontales y separadores

Texto arriba.

Texto medio.

Texto abajo.

---

9. Fórmulas LaTeX

9.1 Display math

$$\int_a^b f(x)\,dx = \lim_{n\to\infty} \sum_{i=1}^{n} f(x_i^*)\,\Delta x$$

$$\frac{d}{dx}\left(\int_a^x f(t)\,dt \right) = f(x)$$

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

9.2 Inline math

La ecuación de Euler $e^{i\pi} + 1 = 0$ es considerada la más bella.

El teorema de Pitágoras: $a^2 + b^2 = c^2$.

---

10. Párrafo largo de resistencia

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione voluptatem sequi nesciunt.

> *"La única forma de hacer un gran trabajo es amar lo que haces." — Steve Jobs*

---

11. Checklist / Tareas

- [x] Título y subtítulo sin asteriscos literales
- [x] Materia e integrantes opcionales en portada
- [x] Imágenes en alta resolución
- [x] Bloques de código con Pygments (monokai)
- [x] Previsualización temporal antes de guardar
- [ ] Soporte para diagramas Mermaid (futuro)
- [ ] Exportación directa a DOCX

---

12. Tabla larga para probar page-break

#	Nombre	Rol	Estado	Observaciones
1	Alice Johnson	Frontend	Activo	Licencia hasta 2027
2	Bob Smith	Backend	Activo	
3	Carol Williams	DevOps	Inactivo	Recontratable
4	Dave Brown	QA	Activo	
5	Eve Davis	PM	Activo	Certificada PMP
6	Frank Miller	Mobile	Activo	
7	Grace Wilson	Data	Inactivo	
8	Henry Moore	Security	Activo	
9	Irene Taylor	Design	Activo	
10	Jack Anderson	Backend	Activo	En período de prueba

Fin del archivo de prueba.